

Programming in Modern C++: Assignment Week 3

Total Marks : 20

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology
Kharagpur – 721302
partha.p.das@gmail.com

August 1, 2025

Question 1

Consider the following snippet.

[MSQ, Marks 2]

```
class Manager {  
    // code  
};
```

```
const Manager mgrObj;
```

What type(s) correctly represent the `this` pointer for object `mgrObj`?

- a) `Manager * const this`
- b) `Manager const * this`
- ☒ c) `Manager const * const this`
- ☒ d) `const Manager * const this`

Answer: c), d)

Explanation:

For a non-const object, the `this` pointer type would be `Manager * const this`, meaning the pointer is constant.

But for a const object like `const Manager mgrObj`, both the object and the pointer are const. Therefore, `this` has the type `const Manager * const this` or equivalently `Manager const * const this`.

~~Question 2~~

Inspect this code block.

[MCQ, Marks 2]

```
class Coordinates {  
    int x, y;  
public:  
    void moveX(int dx) { x += dx; }  
    void moveY(int dy) { y += dy; }  
    void display() const { cout << x << ", " << y; }  
    int shiftX() const { return x + 5; }  
    int shiftY() { return ++y; }  
};
```

Which methods change the object's state?

- a) moveX(), moveY(), display()
- ☒ b) moveX(), moveY(), shiftY()
- c) moveX(), moveY(), shiftX(), shiftY()
- d) moveX(), moveY(), shiftX()

Answer: b)

Explanation:

moveX() and moveY() directly modify member variables. shiftY() changes y via pre-increment. shiftX() returns an expression and does not change state. display() is marked const, and doesn't change anything.

~~Question 3~~

Predict the output.

[MCQ, Marks 2]

```
#include<iostream>
using namespace std;

class Temp {
    int v;
public:
    Temp(int v_ = 0) : v(v_) { cout << "ctor:" << v << " "; }
    ~Temp() { cout << "dtor:" << v << " "; }
};

Temp global(1);
int main() {
    Temp* pT = new Temp(2);
    Temp local(3);
    {
        Temp block(4);
        delete pT;
    }
    return 0;
}
```

- a) ctor:1 ctor:2 ctor:3 ctor:4 dtor:4 dtor:3 dtor:2 dtor:1
- b) ctor:2 ctor:1 ctor:3 ctor:4 dtor:4 dtor:3 dtor:1 dtor:2
- c) ctor:1 ctor:2 ctor:4 ctor:3 dtor:3 dtor:4 dtor:2 dtor:1
- ~~d) ctor:1 ctor:2 ctor:3 ctor:4 dtor:2 dtor:4 dtor:3 dtor:1~~

Answer: d)

Explanation:

Global object `global` is created first (prints `ctor:1`). In `main`, dynamic `pT` is created (`ctor:2`), then local object `local` (`ctor:3`), then block-level object `block` (`ctor:4`). On `delete pT`, destructor runs (`dtor:2`). When block ends, `block` is destroyed (`dtor:4`). When `main` ends, `local` is destroyed (`dtor:3`), then finally `global` (`dtor:1`).

~~Question 4~~

Evaluate the logic and output.

[MCQ, Marks 2]

```
#include<iostream>
#include<deque>
using namespace std;

class Queue {
    deque<char> items;
    int start, end, cap;
public:
    Queue(int size) : start(0), end(-1), cap(size) {
        items.resize(size);
    }
    void enqueue(char val) {
        if (end < cap - 1)
            items[++end] = val;
    }
    void dequeue() {
        if (start <= end)
            start++;
    }
    void display() {
        for (int i = start; i <= end; i++)
            cout << items[i] << " ";
    }
};

int main() {
    Queue q(4);
    for (int i = 0; i < 6; i++)
        q.enqueue(65 + i); // 'A' to 'F'
    q.dequeue();
    q.dequeue();
    q.display();
    return 0;
}
```

- a) C D E F
- b) B C D
- c) C D
- d) C D E

Answer: c)

Explanation:

Max 4 elements allowed. First 4 inserted: A, B, C, D. Then 2 dequeues remove A and B. Remaining are C and D. But display loop shows from index **start** to **end**, so outputs C D.

~~Question 5~~

Observe the following class.

[MSQ, Marks 2]

```
class User {  
    int id;  
    string email;  
    string phone;  
    double balance;  
public:  
    int getId() const { return id; }  
    void setEmail(string e) { email = e; }  
    string getPhone() const { return phone; }  
};
```

Which data members are read-only, write-only, readable and writable, or invisible?

- a) `id` is readonly
- b) `email` is readonly
- c) `phone` is writeonly
- d) `balance` is invisible

Answer: a), d)

Explanation:

`id` has only a getter, so it is readonly. `email` has only a setter, so it is writeonly. `phone` has a getter, so it is readonly. `balance` has neither getter nor setter, so it is invisible.

~~Question 6~~

Analyze the code output.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;
int counter = 5;
class Test {
    int local;
public:
    Test(int val = 0) : local(++val) { ++counter; }
    ~Test() { counter--; }
    void print() { cout << "local = " << local << ", counter = " << counter << endl; }
};

void func() {
    Test t;
    t.print();
}

int main() {
    Test t;
    func();
    t.print();
    return 0;
}
```

- a) local = 1, counter = 6
local = 1, counter = 5
- b) local = 1, counter = 7
local = 1, counter = 6
- c) local = 1, counter = 6
local = 1, counter = 6
- d) local = 2, counter = 5
local = 2, counter = 5

Answer: b)

Explanation:

The first `Test t;` increments `val` to 1 and `counter` to 6. Inside `func()`, another object is created: `counter` becomes 7. Destructor is called at the end of `func()` lowering `counter` to 6.

~~Question 7~~

Evaluate this constructor order.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;
class Demo {
    int a, b, c;
public:
    Demo(int x = 1) : c(x++), b(x++), a(++x) {}
    void print() { cout << a << ", " << b << ", " << c << endl; }
};

int main() {
    Demo d(5);
    d.print();
    return 0;
}
```

- a) 6,6,7
- b) 6,5,7
- c) 6,6,5
- d) 7,6,6

Answer: a)

Explanation:

Order of member initialization follows declaration: a, b, c.

Step-1 Initialize a with ++x → x = 6, a = 6

Step-2 Initialize b with x++ → b = 6, x = 7

Step-3 Initialize c with x++ → c = 7, x = 8

Step-4 So, Final O/P: 6, 6, 7.

Question 8

Analyze and predict the error.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;
class Node {
    int id;
    Node() {} // LINE-A
public:
    Node(int i) : id(i) {}
    void update(Node& n) { this = &n; } // LINE-B
};

int main() {
    Node a(1), b(2);
    a.update(b);
    return 0;
}
```

- a) Error at LINE-A: constructor inaccessible
- b) Error at LINE-B: cannot assign to **this**
- c) Output: no error
- d) Undefined behavior at runtime

Answer: b)

Explanation:

You cannot assign to **this**, as it is a **const** pointer. Hence, LINE-B causes a compilation error.

~~Question 9~~

What will be the output?

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;
class Data {
    int x;
public:
    Data(int x_) : x(x_) { cout << "ctor "; }
    Data(Data &d) { x = d.x; cout << "copy_ctor "; }
    Data& operator=(Data d) { x = d.x; cout << "assign "; return *this; }
};

int main() {
    Data d1(5);
    Data d2 = d1;
    Data d3(10);
    d3 = d2;
    return 0;
}
```

- ~~a) ctor copy_ctor ctor copy_ctor assign~~
b) ctor copy_ctor ctor assign
c) ctor copy_ctor ctor
d) ctor assign copy_ctor

Answer: a)

Explanation:

- Data d1(5); Calls the **constructor** Data(int x_). O/P: ctor
- Data d2 = d1; This triggers the **copy constructor** Data(Data &d). O/P: copy_ctor
- Data d3(10); Calls the **constructor** Data(int x_). O/P: ctor
- In d3 = d2; operation,
 - **Assignment operator** is defined as: Data& operator=(Data d).
 - Argument d is passed **by value**, so this creates a **copy** of d2:
 1. **Copy constructor is called** to create the parameter d, i.e, First, copy_ctor from d2 into temp d. So, O/P: copy_ctor
 2. Then assign is printed in the body of the assignment

Programming Questions

Question 1

Consider the following program. Fill in the blanks as per the instructions given below:

- at LINE-1 with appropriate initialization block to allocate and assign the data members,
- at LINE-2 with appropriate destructor to release memory,
- at LINE-3 and LINE-4 with suitable return statements,

so that the program produces the desired output.

Marks: 3

```
#include<iostream>
using namespace std;

class Coordinates{
    const int *px, *py;
    public:
        Coordinates(int x, int y) : _____{}           //LINE-1
        ~Coordinates(){ _____ }                       //LINE-2
        int getX() const { _____ }                   //LINE-3
        int getY() const { _____ }                   //LINE-4
};

int main(){
    int x, y;
    cin >> x >> y;
    Coordinates c(x, y);
    cout << "[" << c.getX() << ", " << c.getY() << "];"
    return 0;
}
```

Public 1

Input: 7 8

Output: [7, 8]

Public 2

Input: 12 5

Output: [12, 5]

Private

Input: 100 200

Output: [100, 200]

Answer:

LINE-1: px(new int(x)), py(new int(y))

LINE-2: delete px; delete py;

LINE-3: return *px;

LINE-4: return *py;

Explanation:

The constructor dynamically allocates memory for two integers using **new**. The destructor must free that memory. Getters dereference the pointers to return values.

Question 2

Consider the following program. Fill in the blanks as per the instructions given below:

- at LINE-1 with appropriate declaration of a mutable member,
- at LINE-2 and LINE-3 with correct function headers that support const object invocation,

so that the program compiles and produces the correct output.

Marks: 3

```
#include<iostream>
using namespace std;

class Result{
    int m1, m2;
    -----; //LINE-1
public:
    Result(int a, int b) : m1(a*a), m2(b*b){}
    ----- { sum = m1 + m2; } //LINE-2
    ----- {
        cout << "m1 = " << m1 << ", m2 = " << m2 << ", sum = " << sum;
    } //LINE-3
};

int main(){
    int x, y;
    cin >> x >> y;
    const Result r(x, y);
    r.compute();
    r.display();
    return 0;
}
```

Public 1

Input: 2 3

Output: m1 = 4, m2 = 9, sum = 13

Public 2

Input: 4 5

Output: m1 = 16, m2 = 25, sum = 41

Private

Input: 6 7

Output: m1 = 36, m2 = 49, sum = 85

Answer:

LINE-1: mutable int sum

LINE-2: void compute() const

LINE-3: void display() const

Explanation:

Since the object is const, only const functions can be called. sum must be declared mutable to allow modification inside const member function.

Question 3

Consider the following program. Fill in the blanks as per the instructions given below:

- at LINE-1 with a valid copy constructor definition,
- at LINE-2 with a correct copy assignment operator definition,
- at LINE-3 with a condition to check for self-assignment,

so that the program works correctly for dynamic memory handling.

Marks: 3

```
#include<iostream>
#include<cstring>
#include<cstdlib>
using namespace std;

class Student{
    int roll;
    char *name;
public:
    Student(int r, const char* n) : roll(r), name(strdup(n)) {}
    ----- : roll(s.roll), name(strdup(s.name)) {} //LINE-1
    ----- { //LINE-2
        if (-----) { //LINE-3
            free(name);
            roll = s.roll;
            name = strdup(s.name);
        }
        return *this;
    }
    void show(){ cout << roll << " : " << name << endl; }
};

int main(){
    int r1, r2;
    char n1[80], n2[80];
    cin >> r1 >> n1 >> r2 >> n2;
    Student s1(r1, n1);
    Student s2 = s1;
    Student s3(r2, n2);
    s1 = s3;
    s1.show();
    s2.show();
    s3.show();
    return 0;
}
```

Public 1

Input:

1 Alice

2 Bob

Output:

```
2 : Bob
1 : Alice
2 : Bob
```

Public 2

```
Input:
10 John
20 Smith
Output:
20 : Smith
10 : John
20 : Smith
```

Private

```
Input:
3 Eva
4 Lily
Output:
4 : Lily
3 : Eva
4 : Lily
```

Answer:

```
LINE-1: Student(const Student& s)
LINE-2: Student& operator=(const Student& s)
LINE-3: this != &s
```

Explanation:

To avoid memory leaks and ensure deep copy, the class needs to define a copy constructor and assignment opera